

Deep Funding Level III

How We Built Our Dependency Weight Model

A plain-language walkthrough of the problem, what we discovered, and the method behind our submission. No jargon, just the path we took.

The Problem in Plain Terms

Every software project depends on other projects. The contest gives us 83 repositories and asks a simple question: for each one, how much of its value should flow to each of its dependencies?

We put a number (a weight) on every dependency, and for each repository all of its weights have to add up to 1.0. That is 3,677 (dependency, repository) pairs to fill in. A panel of human jurors set the true weights, and our score is how far our numbers are from theirs. Lower is better.

The hard part: we never see the jury's answers for most repositories, so we cannot memorize them. We have to build something that judges well on repositories it has never scored.

Step 1: We Figured Out Exactly How the Score Is Calculated

Most people optimize blind, because the contest never states the exact scoring formula. We pinned it down first, because you cannot improve a number you cannot measure.

The contest gives one small public file of real jury answers (162 pairs across 3 repositories). We used it as a test sheet and tried formulas until our local score matched what the leaderboard reported.

- Our first guess (average error per pair) gave 0.006. Way off, so wrong.
- The formula that matched: for each repository add up the errors of all its dependencies, then average that total across repositories. That gave 0.3440, matching our baseline's leaderboard score of 0.34 to the decimals.

```
score = average over repos of  
      ( sum of |our weight - jury weight| )
```

We then wrote `local_score.py`, an exact copy of the scorer. Now we could test any idea on our own machine before spending a real submission. This was the single most useful thing we did.

Step 2: We Found Where the Error Actually Comes From

With the local scorer we could see which dependencies hurt us most. The answer was clear: in

every repository, the top 6 or so dependencies cause 85 to 100 percent of the total error. The hundreds of tiny dependencies are already close to the jury and barely move the score.

That changed the whole plan. We did not need to fix thousands of numbers. We needed to get the handful of big, important dependencies right for each repository.

Step 3: We Saw What Was Wrong With the Starting Weights

The contest provides baseline weights derived from funding and usage data. Their overall shape is right: small dependencies get small weights, big ones get big weights.

But the head was off. The funding data piles too much weight onto one dependency, and it under-rates libraries that are genuinely critical simply because they did not receive much historical funding. Funding history is not the same as technical importance, and a juror scores on technical importance.

Step 4: Our Fix, an Expert Juror That Corrects the Head

Instead of throwing away the baseline (which is good in the tail), we keep the tail and fix only the head. For each repository we do this:

- Take the top dependencies by baseline weight, since those drive the score.
- Show them, with their baseline weights, to Claude acting as an expert juror.
- Ask one question: how central and irreplaceable is this dependency to what the repository actually does? Core, hard-to-swap pieces get more; easily replaced ones get less.
- Keep the baseline weights for the long tail of small dependencies.
- Normalize so the repository's weights add up to 1.0.

One detail matters for fairness. We do not tell the juror which kinds of dependencies to favor. If we hard-coded patterns from the 3 public repositories, we would just overfit them and likely hurt the 80 we cannot see. So the juror judges every dependency on its own merits, which keeps the method general.

How Well It Works

Scored locally with the exact metric on the 3 public repositories:

Model	Score (lower is better)
Funding baseline	0.344
Rough manual corrections	0.237
Expert-juror head correction	0.121

Then we confirmed it end to end. We uploaded the corrected submission and the public leaderboard returned 0.1206, matching our local 0.121 almost exactly. For context, the cluster of

genuine models near the top sat around 0.18, so our method is comfortably ahead of that group.

Why We Did Not Just Copy the Answer Key

This is the most important thing to understand about our submission.

That public file is also the file the public leaderboard scores against. So anyone can paste those exact numbers and score close to zero on the public board. Many top entries do exactly this. We chose not to, on purpose, because:

- The prize is decided on hidden repositories, where no answer key exists. A copied submission scores near zero on the public 3 and proves nothing about the other 80.
- A pasted answer key shows no method that carries over to unseen data. It is a leaderboard trick, not a model.

So our submission carries the juror's real judgment for all 83 repositories: the 3 public ones land around 0.12 (not zero), and the 80 hidden ones use the exact same method. We would rather be the strongest honest model than the top fake number.

Being Honest About the Limits

We can only directly check 3 of the 83 repositories, because that is all the public answer key covers. Those 3 prove the mechanism works (0.34 down to 0.12). The other 80 use the same method applied fresh. We expect it to carry over, but we cannot verify them directly. This is an informed bet built on a validated mechanism, and we want to say that plainly rather than overclaim.

How to Reproduce

```
pip install -r requirements.txt
python run_ai_juror_pipeline.py
python -c "from src.local_score import score_file;
          score_file('submission_ai_juror_full.csv')"
```

- `claude_juror_corrections.py` holds the juror's head corrections per repo.
- `run_ai_juror_pipeline.py` blends them with the baseline and writes the submission.
- `local_score.py` reproduces the exact leaderboard metric so you can verify it.